

Week 4 Lab Worksheet

The objective of this Week 4 laboratory session was to evaluate the traffic characteristics of heterogeneous IoT protocols and Ad-Hoc routing algorithms. By utilizing **Mininet**, we emulated diverse IoT communication models, capturing and contrasting their transport-layer dependencies (TCP vs. UDP) using **Wireshark (LO1 & LO2)**. Furthermore, we critically analyzed Mobile Ad-Hoc Network (MANET) routing by comparing the periodic control-plane overhead of **Proactive DSDV** against the on-demand Route Request/Reply mechanisms of **Reactive AODV (LO3 & LO4)**.

Learning Objectives

1. Design and emulate heterogeneous Internet of Things (IoT) network architectures.
2. Critically analyse transport-layer dependencies and traffic signatures of diverse IoT protocols.
3. Evaluate and contrast proactive and reactive routing paradigms in Mobile Ad-Hoc Networks (MANETs).
4. Validate control-plane routing behaviours using packet-level traffic analysis.

a. Internet of Things (IoT)

1.1 Message Queuing Telemetry Transport (MQTT)

- a. Navigate to the folder 7COM2008/LAB3 and Right Click and Open in Terminal
- b. Type `sudo python3 iot_lab.py`
 - i. Open two xterms for server and mqtt.
 - a. `xterm mqtt server`
 - ii. On the server xterm, type
 - a. `bash temp_server.sh`
 - iii. On the mqtt xterm, type
 - a. `bash mqtt_sensor.sh`

You will see a temperature simulation taking place between the two IoT nodes.

- c. While traffic is still transmitting, on mininet terminal, open another xterm for server and type
 - i. `sudo wireshark`

`open server-eth0 port on wireshark`
- d. Can you observe MQTT protocol type? Make notes of the similarities observed between MQTT and another protocol you observed last week.
- e. Close all xterm windows

1.2 HyperText Transfer Protocol (HTTP)

- a. Open two xterms for server and http
 - i. `xterm http server`
- b. On the server xterm, type
 - i. `python3 http_server.py`
- c. On the http xterm, type
 - i. `./http_client.sh`
- d. Observe traffic between the two nodes as you did in task 1.1(C)
- e. Close all xterm windows

1.3 Constrained Application Protocol (CoAP)

- a. Open two xterms for server and coap
 - i. `xterm server coap`
- b. On the server xterm type
 - i. `python3 coap_server.py`
- c. On the coap xterm, type
 - i. `python3 coap_client.py`
- d. Observe traffic between the two nodes as you did in task 1.1(C)
- e. Close all xterm windows

1.4 Modicon and Bus (ModBus)

- a. Open two xterms for server and modbus
 - i. `xterm server modbus`
- b. On the server xterm type
 - ii. `python3 modbus_server.py`
- c. On the modbus xterm, type
 - iii. `python3 modbus_client.py`
- d. Observe traffic between the two nodes as you did in task 1.1(C)
- e. Close all xterm windows

1.2 Long Range Wide Area Network (LoRa WAN)

- a. Open two xterms for server and lora
 - i. `xterm server lora`
- b. On the server xterm type
 - ii. `python3 lorawan_server.py`
- c. On the lora xterm, type
 - iii. `python3 lorawan_client.py`
- d. Observe traffic between the two nodes as you did in task 1.1(C)
- e. Close all xterm windows

b. Adhoc Networking

1. Destination-Sequenced Distance-Vector routing (DSDV)

- a. Type `sudo python3 adhoc_topo.py`
- b. Open xterms for all 4 adhoc nodes
 - i. `xterm A B C D`
- c. Type in the following in each respective xterm window
xterm A: `python3 dsdv.py 10.0.0.1`
xterm B: `python3 dsdv.py 10.0.0.2`
xterm C: `python3 dsdv.py 10.0.0.3`
xterm D: `python3 dsdv.py 10.0.0.4`
- d. Open wireshark in Terminal A and observe the traffic pattern.
- e. Press CTRL + C to terminate transmission
- f. Close all xterm windows

2. Ad-hoc On-Demand Distance Vector routing (AODV)

- a. Open xterms for all 4 adhoc nodes
 - i. `xterm A B C D`
- b. Type in the following in each respective xterm window

In xterm B: `python3 aodv.py 10.0.0.2`

In xterm C: `python3 aodv.py 10.0.0.3`

In xterm D: `python3 aodv.py 10.0.0.4`
- c. Trigger the Route Discovery from Node A. Node A wants to talk to Node D, but doesn't know the route yet
- d. Open wireshark from Node A xterm and listen to A-eth0
- e. While listening to A-eth0, in another xterm for node A
 - i. `python3 aodv.py 10.0.0.1 10.0.0.4`
- f. What traffic pattern did you observe in wireshark?

Extension Tasks

1. Compare the connection state and header overhead of MQTT and CoAP. Given that MQTT is widely used in IoT, why might a network engineer choose CoAP (UDP-based) over MQTT (TCP-based) for battery-powered sensors in a highly lossy network (LLN)?
2. In Task 1.4, we simulated Modbus TCP, a protocol heavily used in industrial SCADA systems and PLCs. Critically evaluate the inherent security flaws of the native Modbus protocol. If you were tasked with deploying Modbus sensors in a modern enterprise, how would you secure the network without altering the devices themselves?
3. Unlike HTTP or MQTT, our LoRaWAN emulation encapsulated payloads inside standard UDP packets (Port 1700). Explain the real-world LoRaWAN architecture (End Node → Gateway → Network Server). Why is the physical LoRaWAN Gateway designed to be a "dumb" RF-to-IP forwarder that does not decrypt or terminate the payload?
4. Imagine a scenario where Node A is actively transmitting a large file to Node D via Node C (the intermediate hop). Suddenly, Node C loses power. Contrast how the Proactive (DSDV) and Reactive (AODV) protocols would detect and recover from this mid-session route failure.
5. Both DSDV and AODV rely on network-wide broadcasting (periodic tables for DSDV, RREQ flooding for AODV) to disseminate routing information. Critically evaluate the impact of this mechanism as the network scales from 4 nodes to 4,000 nodes in a dense, static IoT mesh. Which routing paradigm degrades faster, and what specific MAC-layer phenomenon occurs?